TP 4 Part 2

Docker

Objectives:

The goal of this TP is to introduce you to Docker containers and to experiment with it. In this second part, we will see how to deploy containers in Azure.

We will do the following tasks:

- Run a Banana classifier Docker image locally.
- Push it to a container registry.
- Deploy it as a container instance in Azure.
- Deploy it as a container app in Azure and configure an auto-scaling strategy.

Evaluation:

These TPs have two parts that are evaluated:

1. Experimental part: call the Professor to verify what you did when it tells you to do so.
2. Assignment: an assignment needs to be done at the end of the class. Please send it by email to your TP teacher **before midnight the day of the TP**:
    a. Awatif ROUIJEL awatifrouijel@gmail.com
    b. Mohammed SOUIDI souididoc@gmail.com
    c. Yassine EL OUARDI yassine.elouardi@uir.ac.ma
    d. Asmae EL AMRAOUI el.asmae.amraoui@gmail.com

The TP must be done in groups of 2 to 3 students. The groups must stay the same for all the next TPs.

I- Deploy an Azure Custom Vision container and call it:

There is a computer vision service in Azure that allows you to train custom classifiers. This service is called Custom Vision (https://www.customvision.ai/). We used it to create a model that is able to classify ripe from unripe bananas. We then exported the image classification model to a zip file containing a **Dockerfile** (you can find it in Moodle). We also put in Moodle the dataset we used to train this classifier so you can see examples of the two categories.

1. Based on everything you learned during the first part of this TP, and looking at the **README.txt** of the zip file, find how to run the image classification Docker image locally.
2. Based on the **README.txt** of the zip file, check how to call the banana classifier locally.
3. Create an **API query Python program** to call your local image classifier with a banana image URL from the Internet. Hint: you can use the **requests** python module. Check in the documentation of the requests module how to do a **Post** request with custom headers. It should look like this code template (with the … replaced by the correct code):

```
import requests
import json

api_url = …

headers = {
    "Content-Type" : "application/json"
}

body = {
    …
}

response = requests.post(…)
for prediction in response.json()["predictions"] :
    print("Class: ", prediction['tagName'], ", Probability: ",
float(prediction['probability'])*100, "%")
```

4. Call the professor to show him.

II- Publish the custom vision container in a registry:

Before we can run the custom vision container in Azure, we need to publish it to a registry. In the first part of this TP, you saw how to publish the getting started container in Docker Hub. You have to do the same here for the custom vision container. However, we will use here a private container registry from Azure.

1. Based on what you saw in the Share a Docker image part of last TP, push the custom vision container image to a new Docker hub repository called **banana-classifier**.
2. Call the Professor to show him.

III- Run the Custom Vision container in Azure:

1. Create a new resource group in a region available to you called **tp_docker**
2. Create a new container instance in this resource group:



Instances de conteneur

3. Use the following parameters:
   - Name: **banana-classifier-instance**
   - Source: **other registry**
   - Image type: **Private**
   - Image: put here the **name of your Docker hub repository** containing your custom vision container (e.g., okarrakchou/banana-classifier)
   - Image registry login server: **index.docker.io**
   - Image registry user name: **put your docker hub login**
   - Image registry password: **put your docker hub password**
   - Uncheck Azure Spot
   - Operating System: **Linux**
   - Networking:
     - Type of network: **Public**
     - Ports: make sure **TCP port 80** is on the list
   - Leave everything else by default.
4. Call the Professor before clicking on **Create**
5. Go to your container instance and get its Public IP
6. Update the API program you used in section 1 of this TP to now call the container deployed in Azure.
7. Call the Professor to show him.

IV- Make the custom vision container automatically scalable:

You now have run the custom vision container in Azure. We will now create a container app with a scalability rule that will automatically create new instances of the container depending on the number of requests that it receives.

1. Create a new container app in your resource group:

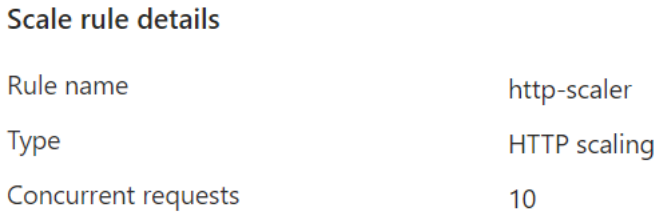


2. Use the following parameters:
   - Name: **banana-classifier-app**
   - Region: **the region where you created your resource group**
   - Container:
     - Uncheck the "Use quickstart image" checkbox
     - Image source: **Docker hub**
     - Image type: **Private**
     - Registry user name: **put your docker hub login**
     - Registry password: **put your docker hub password**
     - Image and tag: put here the **name of your Docker hub repository** containing your custom vision container (e.g., okarrakchou/banana-classifier)
   - Ingress (Entrée): this will create an internal load balancer that will serve requests to your container:
     - Ingress: **Enabled**
     - Ingress traffic: **Accepting traffic from anywhere**
     - Ingress type: **HTTP**
     - Target port: **80**
   - Leave everything else by default
3. Call the Professor before clicking on **Create**
4. Go to your container app and get its URL
5. Update the API program you used in section 1 of this TP to now call the container deployed in Azure.
6. Call the Professor to show him.

V- Change the scalability rule:

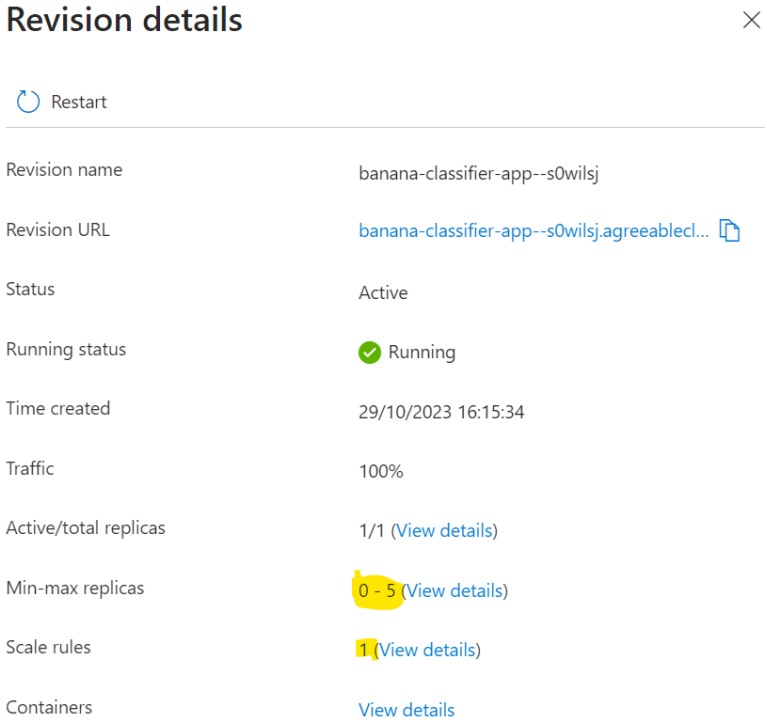
The container app you just created will automatically scale up and down your container depending on the HTTP traffic received. Let's check this:

1. Open the **banana-classifier-app** container app resource you just created in the last section.
2. Click on **Application -> Scale** in the left menu.
3. There will be a scale rule there called **http-scaler**, let's check it. You should see something like this:



By default, the container app will create a new instance of the container every 10 concurrent HTTP requests.

4. Let's now update this scaling rule in order to have a new container instance created for each concurrent HTTP request, with a maximum of 5 instances:
    1. Go back to **Application -> Scale** in the left menu.
    2. Max replicas: put **5**
    3. Scale rule: **Delete http-scaler** and create a new rule with the following configuration:
        1. Name: **my-http-scaler**
        2. Type: **HTTP Scaling**
        3. Concurrent Requests: **1**
    4. Leave everything else by default and save.
    5. Go to **Revisions and replicas** in the left menu and click on your active revision. You should see something like this:



    6. Call the Professor to show him.

VI- Assignment:

Based on what you learned in the TP explain the different steps needed to run a container in Azure. Make sure to explain the different options you use in the commands and take screenshots. Explain the difference between an Azure container instance and an Azure container app.

Do not forget to delete your resource group at the end of the TP!